

Section A: Authentication with Flask-JWT-Extended (10 Questions)

1. What is JWT, and how does it differ from session-based authentication?

JWT (JSON Web Token) is a stateless authentication method where the server generates a signed token after login and the client stores it. The token is sent with every request to prove identity. Unlike session-based authentication, the server does not store user session data. In sessions, user data is saved in memory or a database, while JWT keeps all information in the token itself, making it more scalable.

2. Install and configure Flask-JWT-Extended in a Flask app. Show sample code.

```
pip install flask-jwt-extended

from flask import Flask

from flask_jwt_extended import JWTManager

app = Flask(__name__)

app.config["JWT_SECRET_KEY"] = "secret123"

jwt = JWTManager(app)
```

3. Write a login API that generates a JWT token when username/password are valid.

```
from flask import request, jsonify

from flask_jwt_extended import create_access_token

@app.route("/login", methods=["POST"])

def login():

    data = request.json

    if data["username"] == "admin" and data["password"] == "123":

        token = create_access_token(identity=data["username"])

        return jsonify(access_token=token)

    return jsonify({"msg": "Invalid login"}), 401
```

4. How do you protect a Flask route so that only authenticated users can access it?

```
from flask_jwt_extended import jwt_required

@app.route("/profile")

@jwt_required()

def profile():

    return {"msg": "Welcome, authenticated user!"}
```

5. What is the difference between **access tokens** and **refresh tokens**?

Access tokens are short-lived tokens used for accessing resources, while refresh tokens last longer and are used to request new access tokens without re-login. This improves security.

6. Debug: Your Flask API returns 422: Missing Authorization Header. What's wrong?

This usually means the client didn't send the token properly. JWT tokens should be included in headers like:

Authorization: Bearer <your_token>.

7. How do you store JWT tokens on the client side securely? (Browser/mobile)

In browsers, tokens are stored in HTTP-only cookies to prevent XSS attacks. In mobile apps, they can be stored in secure storage (like Keychain on iOS or Keystore on Android).

8. Write Python code to refresh an expired JWT token.

```
from flask_jwt_extended import jwt_refresh_token_required, create_access_token, get_jwt_identity
@app.route("/refresh", methods=["POST"])
@jwt_refresh_token_required
def refresh():
    current_user = get_jwt_identity()
    new_token = create_access_token(identity=current_user)
    return {"access_token": new_token}
```

9. What are JWT claims? Add a custom claim "role": "admin" in JWT creation.

JWT claims are extra information stored inside the token (like role, email, etc.).

```
create_access_token(identity="user1", additional_claims={"role": "admin"})
```

10. Compare Flask-JWT-Extended with OAuth2 authentication.

Flask-JWT-Extended is simpler and used for custom apps where you control login and tokens. OAuth2 is more advanced, used for third-party logins (Google, GitHub), token scopes, and delegated access. JWT is good for internal APIs, while OAuth2 fits large ecosystems.

Section B: Deployment & CI/CD Pipelines Basics (10 Questions)

11. What is CI/CD, and why is it important in modern software development?

CI/CD stands for Continuous Integration and Continuous Deployment. It automates building, testing, and deploying code so that new changes reach production quickly and reliably. It reduces bugs and speeds up releases.

12. Explain the difference between Continuous Integration, Continuous Delivery, and Continuous Deployment.

CI means developers push code frequently and tests run automatically. Continuous Delivery means code is always in a deployable state but deployment may be manual. Continuous Deployment means every successful build is automatically deployed to production.

13. How do you automate tests in a CI/CD pipeline for a Flask/Django project?

In a CI/CD pipeline for Flask or Django, I write tests using pytest or Django's test framework. Then in the pipeline, I set up Python, install dependencies, and use a test database like SQLite or Postgres. The pipeline automatically runs commands like pytest or python manage.py test on every push or pull request. If tests pass, the code moves to the deployment stage.

14. What is the role of a **build server** in CI/CD?

A build server is responsible for running pipelines. It compiles, tests, and packages code so that it's ready for deployment.

15. Write a simple GitHub Actions YAML workflow to run tests on each push.

```
name: Python Tests
on: [push]
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v2
      - uses: actions/setup-python@v2
        with:
          python-version: "3.9"
      - run: pip install -r requirements.txt
      - run: pytest
```

16. Debug: Your CI/CD pipeline fails at "pip install -r requirements.txt". What should you check?

Check if requirements.txt exists, if the file has proper dependencies, or if Python version mismatch is causing installation errors.

17. How do you handle environment variables (like DB credentials) securely in pipelines?

Use secrets management features provided by CI/CD tools (e.g., GitHub Secrets, GitLab Variables, AWS Parameter Store) instead of hardcoding credentials.

18. What is artifact storage in CI/CD pipelines? Give examples.

Artifacts are build outputs like compiled binaries, Docker images, or test reports. They are stored in repositories like AWS S3, GitHub Packages, or Docker Hub for later deployment.

19. Explain the role of Docker in CI/CD workflows.

Docker ensures that applications run the same way everywhere. In CI/CD, Docker containers are built and tested, then deployed to production without environment mismatches.

20. How would you rollback a deployment if CI/CD pipeline pushes a broken build?

If a broken build gets deployed, I would rollback by redeploying the last stable version. Most CI/CD tools keep previous build artifacts or Docker images, so I can quickly redeploy the last working one. In some setups, I can also use Git to revert the faulty commit and trigger the pipeline again, which restores the stable build.

Section C: Deployment Strategies (Heroku, AWS, etc.) (10 Questions)

21. Deploy a Flask app to **Heroku** using Procfile. Show an example.

create a Procfile:

```
web: gunicorn app:app
```

22. What is the difference between **PaaS** (Heroku) and **IaaS** (AWS EC2)?

Heroku (PaaS) manages infrastructure for you, you just deploy apps. AWS EC2 (IaaS) gives raw virtual machines, and you must manage OS, scaling, and deployment.

23. How do you configure environment variables in Heroku?

```
heroku config:set SECRET_KEY=mysecret
```

24. What is Elastic Beanstalk in AWS? How does it differ from EC2?

Elastic Beanstalk is a PaaS service that manages EC2 instances, scaling, and load balancing automatically. EC2 is just raw virtual machines without automation.

25. Write AWS CLI command to deploy a Docker image to **AWS Elastic Container Service (ECS)**.

```
aws ecs create-service --cluster myCluster --service-name myApp --task-definition myTask
```

26. Compare Heroku vs AWS Lambda for hosting a Python microservice.

Heroku runs long-running apps, while AWS Lambda is serverless and runs only when triggered (cheaper for small workloads, but limited runtime).

27. Debug: Your Flask app crashes on Heroku with H10 App crashed. What are possible causes?

It may be due to missing Procfile, wrong dependencies in requirements.txt, or wrong gunicorn app path.

28. How do you use AWS S3 for static file hosting in a Django/Flask project?

Upload static files to S3 and configure your app to serve URLs from S3 instead of local storage.

29. Explain Blue-Green deployment strategy with an example.

Blue-Green deployment is a strategy where we keep two environments – Blue (current live) and Green (new version). Users are connected to the Blue environment while the Green one is prepared with the new release. Once testing is successful, traffic is switched from Blue to Green, making Green live. Example: Suppose my Django app is running on Blue servers. I deploy the new version on Green servers, test it there, and then switch traffic to Green. If something goes wrong, I can quickly rollback by directing traffic back to Blue.

30. What is Canary deployment, and when should you use it?

Canary deployment means releasing a new version of the app to a small group of users first, while the rest continue using the old version. If it works fine and no major issues are found, we gradually increase traffic until all users are on the new version. You should use it when you want to **reduce risk** and test new features in production with real users but in a controlled way.

Section D: CI/CD with Jenkins & Travis CI (10 Questions)

31. How do you install and configure Jenkins on a server?

I install Jenkins by downloading the .war file or using a package manager (like apt or yum), then run it on Java. After starting, I access Jenkins on port 8080, unlock it with the initial password, install recommended plugins, and create an admin user.

32. What is a Jenkins pipeline, and how is it defined (example Jenkinsfile)?

A Jenkins pipeline is a set of automated steps defined in code to build, test, and deploy applications. It is written in a file called Jenkinsfile using Groovy syntax. Example:

```
pipeline {  
  stages {  
    stage('Build') { steps { sh 'echo Building...' } }  
    stage('Test') { steps { sh 'pytest' } }  
  }  
}
```

33. Write a simple Jenkins pipeline that checks out code, installs dependencies, and runs tests.

```
pipeline {  
  agent any  
  stages {  
    stage('Checkout') {  
      steps { git 'https://github.com/user/repo.git' }  
    }  
    stage('Install') {  
      steps { sh 'pip install -r requirements.txt' }  
    }  
    stage('Test') {  
      steps { sh 'pytest' }  
    }  
  }  
}
```

34. Compare Jenkins with Travis CI.

- Jenkins: Self-hosted, highly customizable, requires setup and maintenance.
- Travis CI: Cloud-based, easier to start, integrates well with GitHub, but less customizable.

35. How do you trigger a Jenkins build automatically when code is pushed to GitHub?

By setting up a webhook in GitHub that notifies Jenkins when code is pushed, and enabling “Build when a change is pushed to GitHub” in job settings.

36. What are Jenkins agents/nodes, and why are they used?

Agents or nodes are machines connected to Jenkins master that run jobs. They are used for distributing builds, running parallel jobs, or running builds on specific environments (e.g., Linux, Windows).

37. Debug: Jenkins build fails at "port already in use". How do you fix it?

Find the process using the port (`lsof -i :8080`), then kill it, or change Jenkins to run on a different port in the configuration.

38. Write a `.travis.yml` file to run tests for a Python project.

```
language: python
python:
  - "3.9"
install:
  - pip install -r requirements.txt
script:
  - pytest
```

39. What are **build badges** in Travis CI, and why are they used in README files?

Build badges are small status icons (e.g., passing/failing) that show the latest build result. They are used in README files to display project health to contributors and users.

40. Compare Travis CI with GitHub Actions. Which one would you prefer and why?

- Travis CI: Simple, cloud-hosted, good for open-source, but limited free minutes.
 - GitHub Actions: Deep GitHub integration, flexible workflows, better free tier.
- I'd prefer GitHub Actions since it integrates directly with GitHub repos and is more modern.